

Visual editing manual

A practical map of the Gyyti website: where each visible section lives, how its visuals are built, and which files and libraries to reach for when making changes.

ONE ROUTE Homepage content is rendered together at /.	IN-PLACE HERO The hero mode changes without navigating away.
INLINE SVG The animated map roads are authored in React.	EDIT HERE FIRST <code>src/routes/index.tsx</code> + <code>src/styles.css</code> .

Prepared from the checked-in source tree. The manual describes the current implementation, not a future mockup. UI labels and content are available in Finnish and English.

Start here: the visual source map

The page is a single React route with styling and copy split into a few focused files. No separate design-tool export or page-per-section structure is used.

What you want to edit	Open this file	Why it is the right place
Section order, markup, labels rendered, links, icon choice, in-place hero	src/routes/index.tsx	Homepage component; all section markup from header through footer.
Colors, type styles, backgrounds, map lines, spacing, responsive layout, motion	src/styles.css	Global design tokens and nearly all named component styles live here.
Finnish / English copy, navigation names, hero and CTA strings	src/lib/i18n.ts	One translation object per language; the route reads the active language from it.
Button look and variants	src/components/ui/button.tsx	Shared button wrapper used by the header, hero and Scope call to action.
Class merging for reusable components	src/lib/utils.ts	cn() joins conditional classes and resolves Tailwind conflicts.
Global fonts, metadata, stylesheet, app shell	src/routes/_root.tsx	Loads styles.css, Google Fonts, favicon, and root providers.
Icons	lucide-react imports in src/routes/index.tsx	Icons are React SVG components; select one in an icon array or JSX.
Map / route artwork	RoadNetwork() in src/routes/index.tsx	Inline SVG paths; styles and animation are in styles.css.
Tab icon	public/favicon.ico	The only standalone visual asset in public/ or src/; no standalone .svg or photo files exist.

Three editing rules that prevent accidental drift

Rule	What to do
Keep the journey on one route	Header links are in-page anchors. The alternate hero is conditional content on the same / route; do not turn it into a second route.
Keep localized strings in sync	Add or edit the same key in both fi and en in src/lib/i18n.ts. Keep punctuation and key structure aligned.
Reuse the existing visual language	Use semantic tokens in styles.css and the Button wrapper instead of hard-coded colors or one-off button markup.

Fastest visual edit: change the matching JSX in src/routes/index.tsx, then adjust its CSS in src/styles.css. If text is user-facing, update both language entries in src/lib/i18n.ts.

Navigation to footer

Every visible homepage section is composed inside the Index component in `src/routes/index.tsx`. The order in this table matches the rendered page.

Order / section	Anchor / key markup	Most important editing points
1. Fixed navigation	<code>header.glass-nav</code>	<code>BrandMark()</code> , desktop anchors, FI/EN switch, store CTA, conditional mobile menu. Anchors target <code>#ominaisuudet</code> , <code>#optimointi</code> , <code>#kayttokohteet</code> , <code>#yhteiso</code> .
2. Hero (mode A / B)	<code>section.hero-stage</code>	<code>RoadNetwork()</code> , map grid, vignette, hero copy, toggle state and store CTA. Keyed inner wrapper replays entrance animation.
3. Partner strip	<code>section.partner-strip</code>	Invitation plus four generated logo placeholders; these are not image files.
4. Product features	<code>section#ominaisuudet</code>	Eight feature cards map translations.items to featureIcons by array index.
5. Route optimization	<code>section#optimointi</code>	<code>VisualPlaceholder</code> plus copy and two metrics. Artwork is generated SVG/CSS.
6. Use cases	<code>section#kayttokohteet</code>	Five rows map translations.items to useCaseIcons by index; clicks currently log an action.
7. Reporting / Scope 1-3	Bordered content section; no id	Reporting copy, outline CTA and second <code>VisualPlaceholder</code> . CTA is a log-action placeholder.
8. Community	<code>section#yhteiso</code>	Colored CTA band, <code>RoadNetwork</code> decoration and external WhatsApp invitation link.
9. Footer	<code>footer.site-footer</code>	Brand summary, store buttons, contact/email, social placeholders and copyright/tagline.

Navigation and calls to action

Header links use ordinary anchors (`href="#..."`) to scroll within the same page. Mobile links also close the menu. The header and hero store buttons call `openStore()`, which delegates platform selection to `storeUrl()` in `src/lib/i18n.ts`. WhatsApp is the one direct external community URL. Several contact/social/use-case/reporting buttons intentionally call `logAction()` only; they do not yet navigate to completed forms or profiles.

The current brand mark is assembled from a Lucide MapPin icon and "Gyyti" text in `BrandMark()`. The larger hero mark combines a MapPin with a letter G. Change JSX for shape/composition and CSS for size, color and glow.

The hero, route art and image placeholders

The site's characteristic map visual is code, not an image asset. Geometry, color and movement have separate edit points.

Visual	Defined in	Important parts / safe edit
RoadNetwork SVG	RoadNetwork() in src/routes/index.tsx	Inline ; roads-static paths draw faint roads, roads-pulse paths animate dashes, route-points circles mark nodes. dense adds an extra static path.
Road line styling	src/styles.css: .road-network, .roads-static, .roads-pulse, .route-points	Stroke uses currentColor / theme variables. Tune stroke-width, opacity, dasharray and route-dash animation.
Map grid and vignette	src/styles.css: .map-grid, .visual-grid, .hero-vignette	CSS gradients and mask create the grid and dark edge treatment. Hero JSX controls stacking.
Optimization / report mockups	VisualPlaceholder() and .visual-placeholder	Calls RoadNetwork(dense), adds grid, status dot and label. Replace with real visuals when approved imagery exists.
Feature / section icons	Lucide imports in index.tsx	Icons render inline as SVG via lucide-react. featureIcons (8) and useCaseIcons (5) are positionally mapped to translated arrays.
Brand marks and UI icons	BrandMark() and route JSX	No custom brand SVG file currently exists. MapPin, menu, arrows, message and other icons are named React components.
Favicon	public/favicon.ico, linked in __root.tsx	Standalone browser-tab icon. Replace the ICO and keep the root link path correct.
Partner logos	Partner-strip JSX	Four ImageIcon placeholders with localized logo labels; replace blocks when real marks arrive.

SVG inventory (important distinction)

Inline SVG: RoadNetwork() contains literal SVG markup in src/routes/index.tsx. Lucide icons also render SVG at runtime. These do not appear as separate .svg files in the repository.

Standalone files: public/favicon.ico is the only image asset under public/ and src/. There are no standalone .svg files, photographs, or raster hero images. robots.txt is a crawler instruction file, not artwork.

Adding real imagery: place approved assets in public/, use an explicit file path from JSX, and replace VisualPlaceholder or partner markup. Keep decorative route art aria-hidden; give meaningful images accessible text.

Hero state and behavior

The first and aspirational autonomous-mobility modes share the same section and CTA. React useState(autonomous) chooses the content branch; the toggle flips it and the keyed inner wrapper replays .animate-stage-in. Metrics are shown only in mode B. This is not a second route or page navigation.

Text, theme and layout controls

These are the most direct places to adjust the look while preserving current component patterns.

Need to change	File / selector	Notes
Finnish or English wording	src/lib/i18n.ts: translations.fi / translations.en	Navbar, hero, feature, use-case, optimization, Scope, community and footer copy. Keep keys matched.
Initial / saved language	src/lib/i18n.ts: detectLang(); index.tsx: lang state	Saved gyyti-lang takes priority; otherwise browser language picks Finnish for fi and English for other locales.
Store button destination	src/lib/i18n.ts: storeUrl()	Uses user-agent detection to choose Apple App Store or Google Play search URL.
Palette / semantic color	src/styles.css: :root and @theme inline	Change oklch variables; utilities like bg-primary / text-accent inherit the theme.
Fonts	__root.tsx Google Fonts links; styles.css font vars	Space Grotesk is display; DM Sans is body. CSS maps them to font-display and body styles.
Spacing / card grids / widths	src/styles.css: .content-section, .feature-grid, .feature-item	Responsive spacing uses clamp(); grid changes at 900px and 560px breakpoints.
Motion	src/styles.css: @keyframes and animation selectors	Stage entrance, route dash, status pulse and partner float. Reduced-motion rules reduce animation.
Shared button appearance	src/components/ui/button.tsx	CVA variants: default/primary/outline/ghost; each call site chooses variant and className.

What the design tokens control

src/styles.css sets the dark navy base, foreground, card / secondary surfaces, primary blue, pale cyan accent, borders and muted text. Tailwind's @theme inline maps these variables to semantic utilities. Prefer changing a shared token over every component. A .dark block is also present; the root currently does not set a dark class, so :root is the visible default palette.

Responsive behavior to preserve

Below 900px, feature cards change from four columns to two, headings stack and hero coordinates hide. Below 560px, features become one column, placeholders shrink, metrics stack and the footer becomes vertical. Header utilities hide desktop links and show the menu below md.

Small implementation notes

Translation objects use as const and should keep matching structure. Feature / use-case icons use array positions, so reordering copy changes its icon unless the matching icon array is also reordered. The route imports ArrowDown but does not render it currently.

Files that own the experience

Visual-editing files are separated from framework plumbing and generated output. The primary route is the safest place to edit the page itself.

File	Role	Important parts
src/routes/index.tsx	Homepage route and visual composition	Route metadata; icons; BrandMark, RoadNetwork, VisualPlaceholder; all navigation, hero, sections and footer; local state / CTA actions.
src/styles.css	Global design system and component CSS	Theme tokens; typography base; layout, section/card/hero selectors; SVG line animation; responsive and reduced-motion rules.
src/lib/i18n.ts	Localization and store URL helper	Lang type; Finnish/English translations; detectLang() and storeUrl().
src/components/ui/button.tsx	Reusable styled button	CVA variants and props shared by route buttons.
src/lib/clsx.ts	CSS class helper	cn() combines clsx and tailwind-merge.
src/routes/_root.tsx	Root HTML shell	Meta, CSS and Google font links, favicon, query provider, error/not-found boundaries, nested route outlet.
src/router.tsx	Router construction	Builds TanStack router and QueryClient; scroll restoration and generated route tree.
src/routeTree.gen.ts	Generated route map	Generated from route files. Do not hand edit.
src/start.ts	TanStack Start middleware	Request error handling and CSRF middleware for server functions; not homepage styling.
src/server.ts	Server entry / SSR errors	Wraps server handler and renders error page on catastrophic SSR errors.
src/lib/error-page.ts, error-capture.ts, lovable-error-reporting.ts	Error reporting and fallback UI	Framework / server error infrastructure; not marketing layout or copy.
src/hooks/use-mobile.tsx	Mobile breakpoint helper	Used by prepared sidebar UI, not by the homepage route directly.
public/favicon.ico; public/robots.txt	Static public files	Browser tab icon and crawler instructions; favicon is the only visual image asset.
components.json	UI generator configuration	shadcn-style aliases and Lucide icon-library setting.
package.json; bun.lock	Package manifest and lockfile	Available dependencies and scripts. Do not edit lockfile by hand.
vite.config.ts	Build configuration	Lovable TanStack Vite preset; configured server entry points to src/server.ts.

Generated file caution: src/routeTree.gen.ts is output. Do not hand-edit the generated tree.

Imported libraries: what they do here

Package presence alone does not mean a library appears on the homepage. This distinguishes live imports from prepared UI-kit dependencies.

Package / source	Why it is here / current use
react, react-dom	Render UI; useState / useEffect power hero mode and language state / effects.
@tanstack/react-router	File-based route at /, metadata, root route and router state.
@tanstack/react-query	QueryClient/provider are wired in router.tsx and __root.tsx; homepage currently makes no queries.
@tanstack/react-start	App start and server middleware / SSR integration.
lucide-react	Named, tree-shaken UI icons; SVGs render inline as React components.
tailwindcss, @tailwindcss/vite	Utility CSS and build-time processing; styles.css imports Tailwind and maps theme tokens.
tw-animate-css	Animation utility stylesheet imported from styles.css.
class-variance-authority	Builds Button variants in button.tsx.
clsx, tailwind-merge	cn() joins conditional classes and resolves conflicting Tailwind utilities.
@lovable.dev/vite-tanstack-config	Vite/TanStack preset supplies React, Tailwind, aliases and server/build plugins.
Google Fonts: DM Sans, Space Grotesk	Remote fonts loaded by __root.tsx for body and display typography.

Installed UI primitives and optional prefabs

These packages support reusable src/components/ui wrappers. The homepage currently imports only button.tsx from this directory; other files are prepared building blocks, not visible marketing sections.

Group	Installed packages	Where / why
Radix primitives	@radix-ui/react-accordion, alert-dialog, aspect-ratio, avatar, checkbox, collapsible, context-menu, dialog, dropdown-menu, hover-card, label, menubar, navigation-menu, popover, progress, radio-group, scroll-area, select, separator, slider, slot, switch, tabs, toggle, toggle-group	Headless accessibility / keyboard behavior used by corresponding UI wrappers. Homepage Button uses CVA, not Radix.
Form / date / command	@hookform/resolvers, react-hook-form, zod, react-day-picker, date-fns, input-otp, cmdk	Form validation, date-picker, OTP and command-menu prefabs; not used by homepage sections.
Display / interaction	recharts, embla-carousel-react, react-resizable-panels, vault, sonner	Chart, carousel, resizable-panel, drawer and toast prefabs.
Build / tooling	vite, @tanstack/router-plugin, vite-tsconfig-paths, TypeScript, ESLint, Prettier, Nitro	Build, route generation, TS path aliases, checks and server output; dev dependencies.

Check package.json for exact versions. Add packages only when implementing a feature that needs them; keep dependencies and bun.lock in sync.

The prepared src/components/ui files

These wrappers are available for future UI, but are not all part of the visible marketing page. Button is the only file here imported by the homepage.

Component file / purpose	Component file / purpose	Component file / purpose
accordion.tsx Expandable content groups	alert-dialog.tsx Confirmation / destructive-action dialog	alert.tsx Inline status and alert messages
aspect-ratio.tsx Fixed-aspect-ratio media container	avatar.tsx User portrait / initials	badge.tsx Compact status or category label
breadcrumb.tsx Hierarchical navigation trail	button.tsx Styled button with variants; used	calendar.tsx Calendar / date-picker UI
card.tsx Generic content surface	carousel.tsx Carousel controls and Embla integration	chart.tsx Chart container / Recharts integration
checkbox.tsx Checkbox input primitive	collapsible.tsx Disclosure panel	command.tsx Searchable command palette
context-menu.tsx Right-click context menu	dialog.tsx Modal dialog	drawer.tsx Drawer sheet / mobile panel
dropdown-menu.tsx Dropdown action menu	form.tsx React Hook Form field wrappers	hover-card.tsx Hover-triggered detail card
input-otp.tsx One-time-password input	input.tsx Text input	label.tsx Form label
menubar.tsx Application menu bar	navigation-menu.tsx Structured navigation dropdown	pagination.tsx Paged-list navigation
popover.tsx Anchored popover	progress.tsx Progress indicator	radio-group.tsx Single-choice radio controls
resizable.tsx Resizable panels	scroll-area.tsx Styled scroll container	select.tsx Select / dropdown input
separator.tsx Visual divider	sheet.tsx Dialog-based side sheet	sidebar.tsx Responsive sidebar composition
skeleton.tsx Loading placeholder	slider.tsx Range slider	sonner.tsx Toast notification wrapper
switch.tsx On/off control	table.tsx Table primitives	tabs.tsx Tabbed panels
textarea.tsx Multiline text input	toggle.tsx Single pressed/unpressed control	toggle-group.tsx Grouped toggle controls
tooltip.tsx Hover/focus hint		

Additional shared pieces: src/hooks/use-mobile.tsx supports responsive sidebar behavior; src/lib/utils.ts provides cn(); components.json documents aliases and the Lucide icon setting. These pieces support the UI kit, not homepage sections.

Common visual edits, step by step

Use these paths when you know what to change and need the right file quickly.

Goal	Edit path
Change the hero headline	src/lib/i18n.ts -> translations.fi.hero.title1/title2 and translations.en equivalents.
Change the two hero modes	src/routes/index.tsx -> conditional autonomous ? ... : ... block; copy lives in i18n.ts. Keep one route.
Move / rename a section	Update JSX order or id in index.tsx and the matching header href plus both translated nav labels.
Change map path geometry	Edit path d coordinates and circle coordinates in RoadNetwork() in index.tsx; tune strokes / motion in styles.css.
Change page palette	Edit semantic tokens under :root in styles.css. Check contrast for hero, community band, buttons and muted text.
Add / swap an icon	Import from lucide-react. Change featureIcons / useCaseIcons in the same position as the translated label.
Replace placeholders	Add asset to public/ and replace VisualPlaceholder or partner-float JSX. Add meaningful alt text and preserve aspect ratio.
Change mobile navigation	index.tsx: menuOpen and mobile-menu; styles.css: .mobile-menu; visibility also uses Tailwind breakpoint utilities.
Change a reusable CTA style	button.tsx for a new variant; index.tsx for selecting it at a call site.
Wire a placeholder action	Replace the relevant logAction() callback in index.tsx with the real URL / form flow and accessible semantics.

How to read the component file

Index is composed from local components (BrandMark, RoadNetwork, VisualPlaceholder) plus inline section JSX. Repeated content is data-driven: features and use cases map translated arrays to icon arrays. CSS class names connect markup to styles.css; Tailwind utilities handle common layout and spacing.

What is intentionally not a separate page

The second hero is a visual state inside the same homepage. Navigation anchors scroll to sections, and the main CTA opens the platform-specific app-store search. Keep these mechanics distinct when editing copy or adding links.

Visual editing checklist

Before considering a visual edit complete, check the pieces that tend to be split across files.

- Markup: correct section changed in index.tsx; section order and anchor IDs still make sense.
- Copy: Finnish and English translations updated together in i18n.ts.
- Style: semantic colors and responsive CSS remain consistent in styles.css.
- Icons: positional icon arrays still align with translated feature / use-case arrays.
- Assets: SVG is intentionally inline, or new public asset path exists; meaningful imagery has accessible text.
- Navigation: mobile menu, in-page links, and same-route hero toggle remain intact.
- Buttons: preserve Button variants; replace console.log placeholders only when a real action is specified.
- Responsive / motion: check narrow layout and retain reduced-motion support.

Run the existing project commands

Development: `bun run dev` (or package-manager equivalent).

Production build: `bun run build`.

Lint: `bun run lint`.

Scripts are defined in package.json. The app uses Vite with a TanStack Start server build and generated file-route tree.

Best starting point: open `src/routes/index.tsx` and `src/styles.css` side by side. Treat `src/lib/i18n.ts` as the source of truth for Finnish / English copy. Route artwork is authored inline; there is no external SVG/photo library.

Manual scope: visual structure, related files and dependency roles found in the current repository source. Exact package versions are in package.json and bun.lock.